

Chayma ROUAA 28706408
Mélany SOUTO 28604997

L2 Mono Mécanique
LU2ME005-F
Groupe 4-A/B

RAPPORT DE PROJET

MODÉLISATION D'UN ÉCOULEMENT DE SABLE

En FORTRAN 90



ANNEE 2021-2022

SOMMAIRE

SOMMAIRE	2
INTRODUCTION	3
PROGRAMMATION	4
RÉSULTATS	6
AMÉLIORATIONS	7
CONCLUSION	8

INTRODUCTION

Nous nous proposons dans cette étude de modéliser et simuler un écoulement de sable sur Fortran 90.

L'objectif de ce projet est de voir un tas de sable (écoulement granulaire) se former au fur et à mesure sur une durée de temps déterminée. La première version de notre projet nous a permis de visualiser la «moitié» d'un tas, dont le sommet h_{max} se trouve à l'extrémité gauche de l'écran lors de l'animation et dont le rayon r_{max} représente le contact au sol. h_{max} et r_{max} sont saisies par l'utilisateur et vérifiées par une lecture contrôlée. Nous avons pu repartir de notre première version en simplifiant certains éléments, et poursuivre vers la version finale.

La deuxième version fait agir l'utilisateur. Il peut choisir ou non d'afficher le tas de sable.

Les résultats obtenus sont sous la forme d'un tableau de vecteurs nommé grille qui s'efface à chaque pas de temps pour donner un effet plus intuitif et réaliste d'écoulement. Les vecteurs sont nommés pile, et représentent donc les piles verticales de sable.

Nous avons pu résoudre certains soucis rencontrés notamment au niveau de l'affichage de l'écoulement qui ne se faisait pas dans le bon sens ; il nous fallait tourner la tête à 90° pour pouvoir observer correctement la chute de sable. Un autre était lié à la boucle de temps (allant de 0 à temps max par pas de nt) ; il nous a fallu lui donner une valeur finale par défaut pour que le programme puisse fonctionner.

PROGRAMMATION

Le programme fortran utilisé pour notre projet va être détaillé ci-dessous. La première étape consiste à afficher la moitié d'un écoulement de grains correspondant à la première version rendue. On supposera que chacune des variables traitées ont bien été déclarées dans le bloc déclaratif.

Dans un premier temps on réalisera un module `m_sable` contenant dans la partie déclarative `IMPLICIT NONE`, un type dérivé "tas" pourvu de 4 champs : `rmax` et `hmax`, qui représentent des entiers, ainsi que `pile` et `grille`, deux tableaux dynamiques. Le premier est à une dimension et représente le nombre de grains par numéro de pile, tandis que le second est à deux dimensions et permettra de représenter le tas de sable par le moyen de caractère comme des points "." (chaque point représentant un seul grain de sable).

Une fois le module saisi, on crée le programme `p_sable` dans lequel on demandera à l'utilisateur de saisir la hauteur `hmax` et le rayon `rmax` souhaités par le biais d'un appel à une procédure, placée dans le module `m_sable`. Cette fonction, appelée "lecture_controlée", prenant comme paramètre `nmin` et `nmax`, permettra à l'utilisateur de saisir une valeur de `hmax` ou de `rmax` tout en vérifiant qu'elle soit bien comprise entre les 2 valeurs limites (ici 3 et 40) . Cette vérification se fera grâce à une boucle `DO` à condition de sortie `IF EXIT` correspondant à la bonne appartenance de la valeur à saisir à l'intervalle `nmin nmax`. Cette boucle servira à demander la saisie par l'utilisateur jusqu'à ce que la saisie soit correcte, répétant la question à chaque saisie incorrecte.

On alloue ensuite quatre tableaux, grâce aux valeurs saisies: `mon_tas%pile`, `mon_tas%grille` (`mon_tas` étant une variable de type `tas` déclarée dans le programme), `distrib_taille` (correspondant au nombre d'avalanches par taille), et `n_avalanche` correspondant au nombre d'avalanche à chaque ajout de grain. Par la suite on initialise le tableau `mon_tas%pile` à 0 et l'entier `i` à 0. (qui servira comme compteur de boucle.)

Pour permettre le remplissage du tas de sable, on réalise une première boucle appelée boucle de temps allant de 0 à `temps_max` (initialisé à 10 000) par pas de `nt` (initialisé à 10). Elle permettra d'ajouter un grain de sable à la pile de sable 0 (`mon_tas%pile(0)`) toutes les 10 unités de temps (permettant donc une capacité de 1000 grains de sables). Cet ajout sera fait seulement si `mon_tas%pile(0)` est inférieur à la hauteur `hmax` saisie par l'utilisateur (condition codée par l'intermédiaire d'un `IF`).

Dans cette première boucle de temps, et avant l'ajout d'un grain à la pile 0, on créera une nouvelle boucle `DO` utile à l'opération de transfert de grain, allant par pas de 1 (`i = i+1` à chaque tour). Le nombre de grain transféré sera obtenu grâce à la formule suivante :

$$ng = 1 + \lfloor 1/2 (2 + pile(i) - pile(i+1)e \rfloor$$

Avec `e` un nombre aléatoire (dans `[0, 1[`) , obtenu grâce à l'appel à la subroutine `RANDOM_SEED` en début de programme, puis l'appel à la subroutine à l'intérieur de

la deuxième boucle RANDOM_NUMBER(e). On transfère un nombre ng de grains de sable d'une pile i vers la pile i+1(la suivante) , sans oublier de retirer à cette pile i le nombre ng de grains que nous avons ajouté à l'autre.

Cette boucle aura comme condition de sortie IF EXIT. Condition atteinte soit lorsque mon_tas%pile(i) sera égale au nombre de pile max-1 (donc mon_tas%(rmax-2) car nombre de pile max est rmax-1) soit lorsque la hauteur d'une pile sera supérieure ou égale à la hauteur max saisie par l'utilisateur.

A chaque changement de taille de la pile i, on appellera une subroutine "affiche" placée dans le module m_sable. Cette subroutine donne les instructions nécessaires pour remplir la grille d'espaces " " (pour donner l'impression de vide) ou de points "." (pour représenter les grains de sable), par l'intermédiaire de deux boucles dans une boucle. La boucle extérieure balayera chacune des piles de grains dans l'ordre, tandis que les deux boucles internes, rempliront les piles de ' ' ou de '.' . Cette subroutine contient une dernière boucle qui permettra l'affichage du tableau grille à travers un PRINT.

Pour la deuxième version nous avons remplacé le modèle standard de saisie de la hauteur et du rayon max, par un modèle de lecture de fichier en rentrant les données voulues dans un fichier appelé "param.dat" qu'on ouvre grâce à la commande OPEN en début de programme. Une fois le fichier ouvert on lit et initialise chaque valeur grâce à la commande READ, puis on referme param.dat.

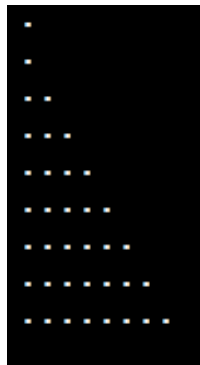
Concernant la partie avalanches, on ajoute 1 au nombre d'avalanche n_avalanche dès qu'il y a un changement de nombre de grains dans une pile, puis on ouvre un fichier distrib_taille dans lequel on remplit un tableau du même nom représentant le nombre d'avalanches en fonction de leur taille grâce à une boucle DO allant de 1 au rayon maximum de la pile.

On ouvre également un fichier tas_final.res dans lequel on remplit le tableau du même nom avec comme première colonne, le numéro de pile, et deuxième colonne le nombre de grains correspondant par le biais d'une boucle DO.

RÉSULTATS

Voici l'affichage du tas de sable mentionné dans PROGRAMMATION.

Le tas n'est représenté qu'à moitié avant amélioration.



Le sommet $h_{\max}$ se situe à gauche, et $r_{\max}$ correspond au nombre de piles, soit au nombre de grains dans la première ligne.

Le programme fonctionne tel qu'on le souhaitait. Le tas en deux dimensions forme un demi tas. Cependant, même lorsqu'on met les mêmes paramètres dans le fichier `param.dat`, le résultat peut être différent. Cela est dû au caractère aléatoire du nombre de grains transférés. La modélisation reste tout de même cohérente.

Figure 1. Tas de sable de la version 1 au temps final

Les données entrées pour ce tas dans `param.dat` sont les suivantes :

```
1 12      rmax
2 9       hmax
3 o       affichage du tas (o/n) ?
4 tas_final.res  nom du fichier
```

Figure 2. données du fichier param.dat

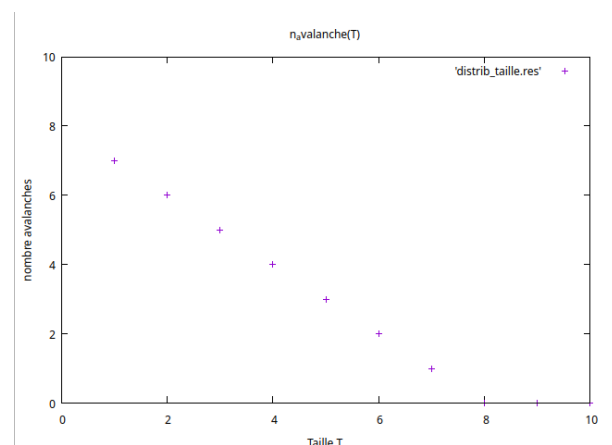
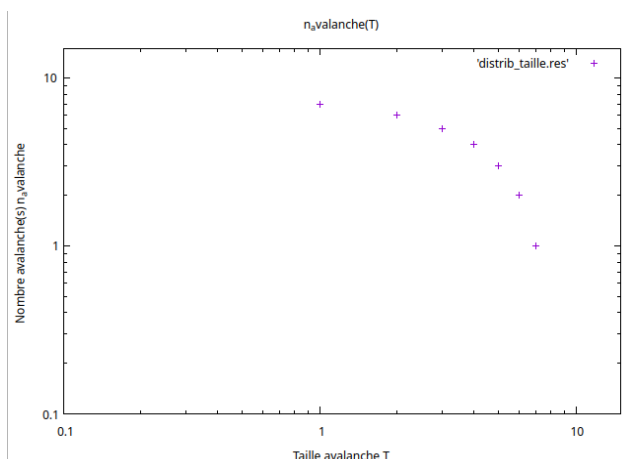
La version finale, plus aboutie, affiche un tas où la pile atteignant $h_{\max}$ se trouve au centre du tas, nous la présenterons dans la partie dédiée aux améliorations.

Concernant l'exécution du code, le résultat est assez satisfaisant avec une animation représentant la formation du tas de sable en direct obtenue grâce à l'appel aux sous-routines "system ('clear') " et " system ('sleep 0.1') " .

On note également la bonne ouverture des fichiers `distrib_tailles.res` et `tas_final.res`.

Voici le tableau représentant le nombre d'avalanches en fonction de la taille

Figure 3. Graphe représentant le nombre d'avalanches en fonction de leur taille en échelle log-log (à gauche) et en représentation linéaire (à droite)



Pour le modèle BTW, nous obtenons la même distribution. Nous remarquons que plus la valeur de $h_{\max}$ est grande plus la distribution augmente.

Nous avons ensuite un graphe 'un_tas.res' traçant le numéro de pile en fonction du nombre de grains. La courbe du graphique semble représentative du tas qui s'affiche à l'écran, bien que non linéaire.

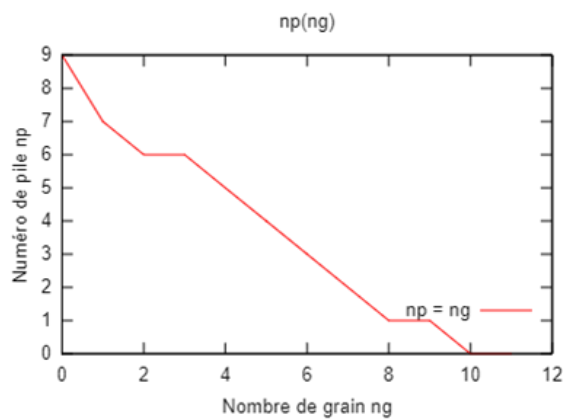


Figure 4. Graphe représentant le numéro de la pile en fonction du nombre de grain

Il s'agit là des données pour la moitié d'un tas, le tas entier nous le rappelons sera abordé plus tard.

AMÉLIORATIONS

Concernant les améliorations nous avons décidé dans un premier temps de réaliser la première proposition d'amélioration, c'est-à-dire la réorganisation du code de manière à ce que le programme principal nommé `p_sable` ne contienne que des appels à des procédures placées dans le module "`m_sable`".

Pour atteindre notre objectif nous avons trois possibilités.

La première étant de réaliser trois sous-routines dans le module, une première d'initialisation, codant pour l'initialisation des valeurs, soit l'ouverture et la lecture du fichier "`param.dat`". La deuxième contenant les boucles principales codant pour le remplissage du tas de sable ainsi que le nombre d'avalanches et le remplissage du tableau `distrib_taille` (contenant le nombre d'avalanche en fonction de leur taille). La troisième sous-routine contenant l'ouverture des fichiers `distrib_tailles` et `tas_final.res` ainsi que leurs remplissages. Cette méthode avec trois sous-routines impliquait de déclarer plus de variables dans le programme principal, ce qui nous a poussé à nous tourner vers la deuxième possibilité qui s'offrait à nous.

La seconde étant de tout regrouper en une seule sous-routine impliquant la déclaration de seulement deux variables dans le bloc déclaratif du programme principal `p_sable`: "`mon_tas`", variable de type `tas`, type déjà présent dans le module `m_sable`, ainsi que "`ecran`", un caractère qui permettra de déterminer si oui ou non l'utilisateur souhaite un affichage à l'écran. Cette solution était celle permettant de simplifier un maximum le programme principal, d'où notre choix.

Nous aurions pu également séparer cette sous-routine en deux, une contenant l'initialisation et le remplissage du tas de sable, l'autre contenant l'ouverture des fichiers `tas_final.res` et `distrib_tailles` (ainsi que leurs remplissages). Toutefois, cette option impliquait, comme la première possibilité, une déclaration plus importante de variables.

Une fois la sous-routine créée, on sépare en deux fichiers le module (`m_sable.f90`) et le programme principal (`p_sable.f90`) qu'on compilera séparément. On tapera dans un premier temps dans le terminal "`gfortran -c m_sable.f90`", puis "`gfortran -c p_sable.f90`", et enfin "`gfortran -o prog m_sable.o p_sable.o`".

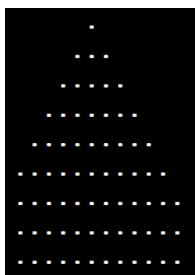
Il ne reste ensuite qu'à exécuter le programme.

Nous avons également décidé de réaliser la deuxième proposition d'amélioration, c'est-à-dire la simulation d'un tas de sable entier et non d'une moitié de tas.

Pour atteindre notre objectif nous avons deux possibilités.

La première consistait simplement, à ajouter dans la subroutine affichage une boucle Do balayant dans le sens inverse le tableau `un_tas%grille`, permettant de réaliser un tas parfaitement symétrique au tas déjà obtenu. Ce modèle là contient toutefois de nombreux désavantages : puisque le tableau représentant le tas final sera composé de deux tableaux symétriques, le rayon du tas final sera toujours le double de celui entré par l'utilisateur (ou écrit dans le fichier `param.dat`). Le deuxième inconvénient est l'implication que deux grains tombent en même temps à chaque pas de temps. C'est en raison de ces deux problèmes que nous nous sommes penchés vers la deuxième possibilité.

La deuxième consiste à ajouter à chaque boucle de temps un grain à la pile `mon_tas%rmax/2 + 1` (correspondant à la pile du milieu du tas) plutôt qu'à la pile 0. On ajoute ensuite au programme une constante `k` initialisée à 1 qui augmentera de 1 à chaque fois qu'un grain tombe. Dans la boucle de temps on ajoute la condition que `k` est pair. Si `k` est pair alors on réalise la boucle de transfert de grain de `i` à `i+1` en partant de `i = mon_tas%rmax/2 + 1`, soit à la moitié de la pile (donc transfert du grain vers la droite). Si `k` est impair alors on réalise le transfert de grain de `i` à `i-1` (transfert vers la gauche) avec la même pile de départ et comme pile d'arrivée du transfert, la pile 0. Dans ce cas là on fera à chaque boucle `i = i-1`. Le défaut de cette modélisation est que le tas est peu symétrique pour des valeurs de `rmax` paires.



Ceci est un exemple de tas obtenu lorsque l'on rentre une valeur de `rmax` à 12, et une valeur de hauteur max à 9.

Il n'est pas parfaitement symétrique, cela est à la fois du au caractère aléatoire des transferts de grains et au fait que le rayon `rmax` soit pair.

Figure 5. tas de sable final obtenu grâce au code après amélioration.

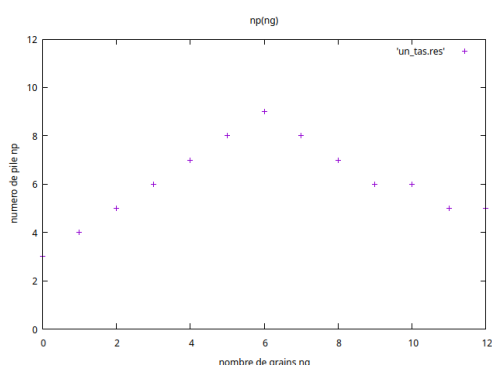


Figure 6. Graphe représentant le numéro de la pile en fonction du nombre de grain

On remarque que les points du graphique semblent représenter le tas de sable obtenu. Le résultat est bien cohérent.

CONCLUSION

Finalement, nous avons réussi à modéliser en deux dimensions la formation d'un tas de sable dont les dimensions sont choisies par l'utilisateur. L'écoulement au cours du temps est satisfaisant et correspond à nos attentes. Nous obtenons, selon les données entrées dans le fichier param.dat, un affichage fluide qui montre une évolution plus ou moins cohérente du tas pendant la durée de temps temps_max définie.

Cependant on pourrait s'interroger sur les limites de ce modèle. En effet, dû à la gravité et le comportement de la matière dans ce cas physique précis, un tas de sable, n'est pas censé avoir les côtés aussi raides que nous constatons pour le nôtre. Il serait plus réaliste d'obtenir un tas plus "aplati". Ainsi, l'inclinaison des arêtes (fictives) du tas de sable serait moins raide mais donc les arêtes seraient également moins lisses. Il faudrait donc peut-être donner à rmax une valeur toujours plus grande que hmax pour avoir cet effet "aplati", et changer alors dans le programme la condition imposant l'arrêt d'ajout de grains dans le tableau dès lors que hmax est atteint.

Pour aller plus loin, il aurait été intéressant de modéliser l'écoulement de sable dans un sablier. Pour cela, nous pourrions envisager de procéder de cette manière par exemple : un premier tableau de même dimensions que mon%tas%grille représentant des grains de sable formant un triangle isocèle dont le sommet serait sur la dernière ligne et la base sur la première ligne. Les grains "tomberaient", donc disparaîtraient du tableau, et on fait apparaître dans un deuxième tableau, affiché à la suite (en dessous), notre tableau dynamique mon%tas%grille en prenant garde à ce que le grain qui sort du premier tableau se place à la même colonne (soit montas%pile(i)) que le grain qui apparaît (donc entre) dans le tableau montas%grille.

On prendrait bien en compte que le nombre de grains reste constant pour les deux tableaux, c'est-à-dire que le nombre de grains à l'instant 0 du premier tableau soit égal au nombre de grains des deux tableaux à tout instant. Autrement dit, pour chaque grain sortant du premier tableau, nous observons un grain apparaître dans le deuxième.